CODE ANALYSIS IN GHIDRA, FINDING BUGS AND APPLYING A PATCH TO CODE BINARIES, AND VULNERABILITY ASSESSMENT USING OWASP ZAP PROXY

Hashan Kodippilige

Software Security Analysis - Project 2

INTRODUCTION

- Focus on the analysis of software security through reverse engineering and vulnerability assessment techniques.
- Identify how compiled binaries operate without access to source code and identify potential flaws.
- Ghidra and IDA Pro are used for the examination of program structure, functions, and logic at both assembly and pseudocode levels.
- Reconstruct the internal behavior of the programs to identify logical errors and implementation issues.
- Modify assembly instructions to correct the program error by applying patches to a binary to fix identified bugs.
- OWASP ZAP (Zed Attack Proxy)used to perform automated scanning of a test website to identify security weaknesses

GOALS

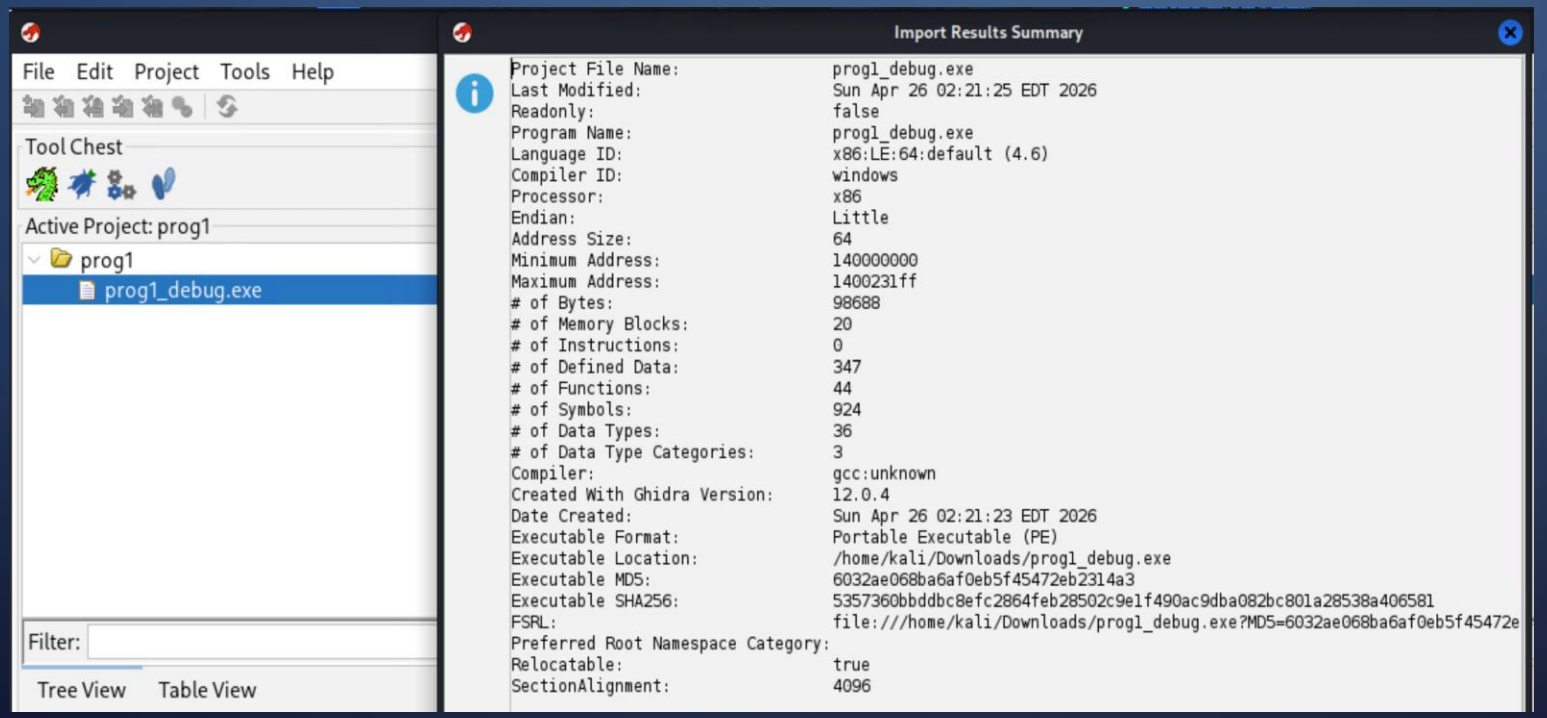
- Identify bugs
- Fix binaries
- Detect web vulnerabilities

TOOLS USED

- **Ghidra** – Reverse engineering
- **IDA Pro** – Binary analysis & patching
- **OWASP ZAP** – Web vulnerability scanner
- **Kali Linux** – Testing environment

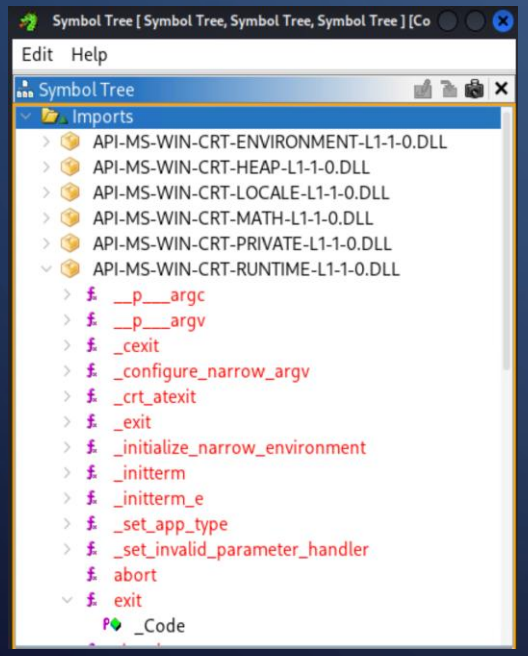
PART 1- CODE ANALYSIS IN GHIDRA

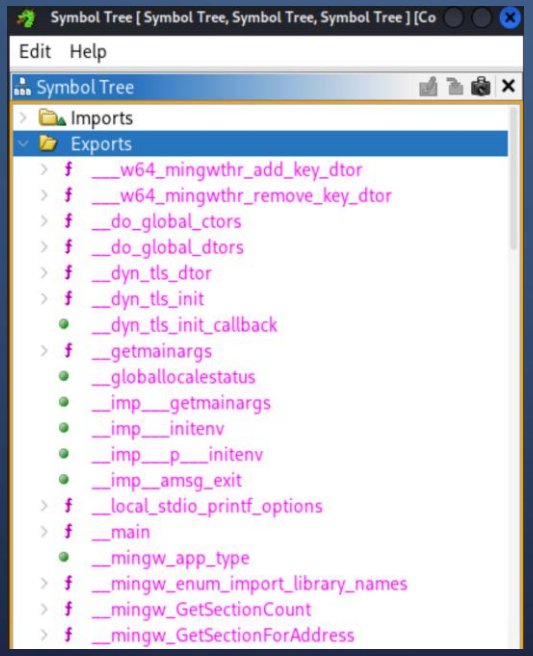
- Imported .exe file
- Performed automatic analysis

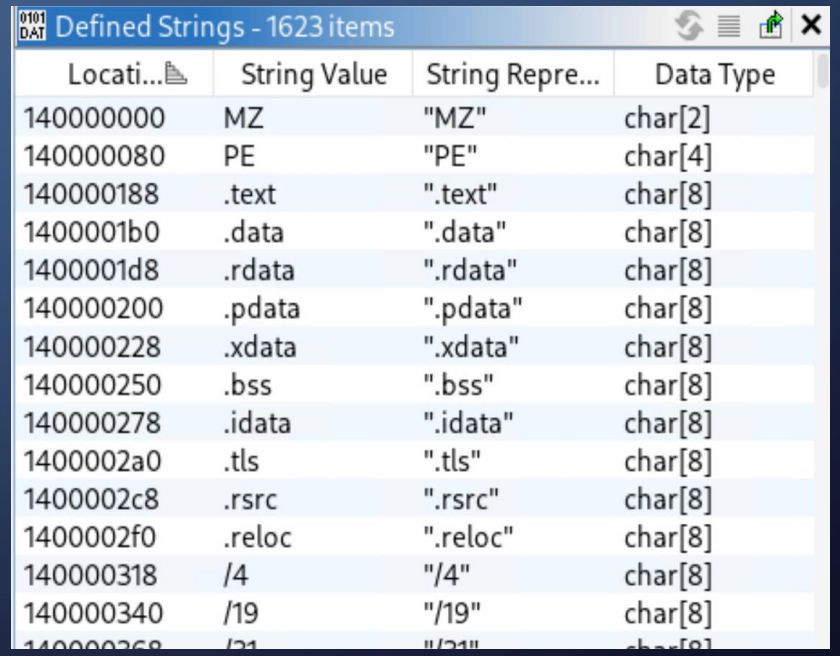


CODE ANALYSIS IN GHIDRA

- Explored:
  - Imports & exports functions
  - Strings







Locati...	String Value	String Repre...	Data Type
140000000	MZ	"MZ"	char[2]
140000080	PE	"PE"	char[4]
140000188	.text	".text"	char[8]
1400001b0	.data	".data"	char[8]
1400001d8	.rdata	".rdata"	char[8]
140000200	.pdata	".pdata"	char[8]
140000228	.xdata	".xdata"	char[8]
140000250	.bss	".bss"	char[8]
140000278	.idata	".idata"	char[8]
1400002a0	.tls	".tls"	char[8]
1400002c8	.rsrc	".rsrc"	char[8]
1400002f0	.reloc	".reloc"	char[8]
140000318	/4	"/4"	char[8]
140000340	/19	"/19"	char[8]

CODE ANALYSIS IN GHIDRA (ANTI-DEBUGGING)

- Found function:
  - is_debugger_present()
- Behavior:
  - If debugger detected - program exits
  - If not - continues execution
- Purpose - prevent reverse engineering

Decompile: main - (prog1_debug.exe)

```
1
2  /* WARNING: Unknown calling convention -- yet parameter storage is locked */
3
4  int main(void)
5
6  {
7    int iVar1;
8
9    __main();
10   iVar1 = is_debugger_present();
11   if (iVar1 == 0) {
12     puts("None detected. Running safely.");
13     Sleep(2000);
14     puts("Keylogger has completed an operation.");
15   }
16   else {
17     puts("Debugger detected! Exiting.");
18   }
19   return (int)(iVar1 != 0);
20 }
21
```

PART 2- FINDING THE BUG

- Analyzed binary in IDA Pro
- Focused on main() function
- Found issue in calculation:
  - Total = amount – tax
  - Total should be amount + tax

```
int __fastcall main(int argc, const char **argv, const char **envp)
{
  float v3; // xmm0_4
  float v5[2]; // [rsp+Ch] [rbp-14h] BYREF
  float v6; // [rsp+14h] [rbp-Ch]
  unsigned __int64 v7; // [rsp+18h] [rbp-8h]

  v7 = __readfsqword(0x28u);
  printf("\nEnter in the amount purchased: ");
  __isoc99_scanf("%f", v5);
  v3 = 0.075 * v5[0];
  v5[1] = v3;
  v6 = v5[0] - v3;
  printf("\nThe sales tax is $%4.2f", v3);
  printf("\nThe total bill is $%5.2f\n", v6);
  return 0;
}
```

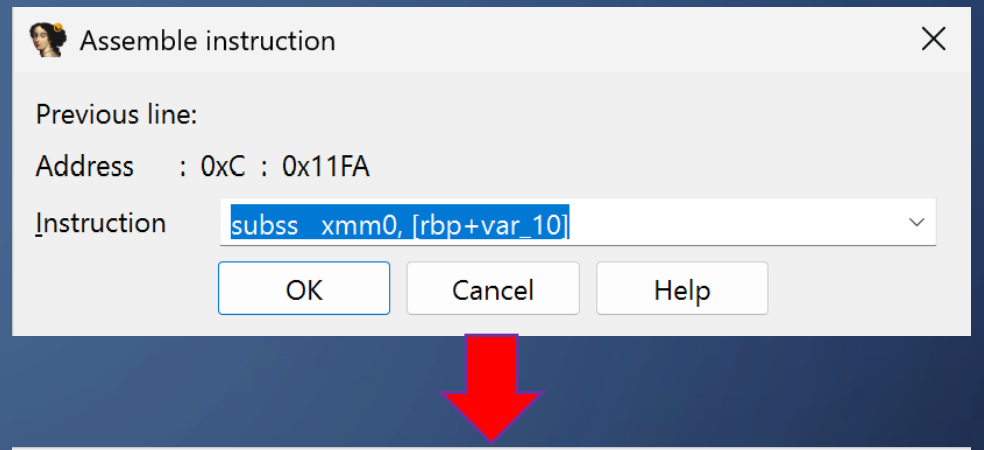
BUG EXPLANATION – KEY FINDINGS

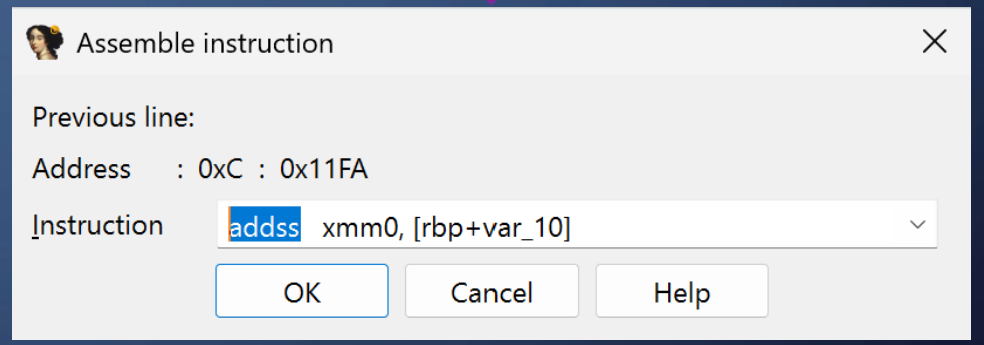
- Tax = 7.5% of input
- Incorrect logic:
  - Subtraction used instead of addition
- Leads to wrong total bill

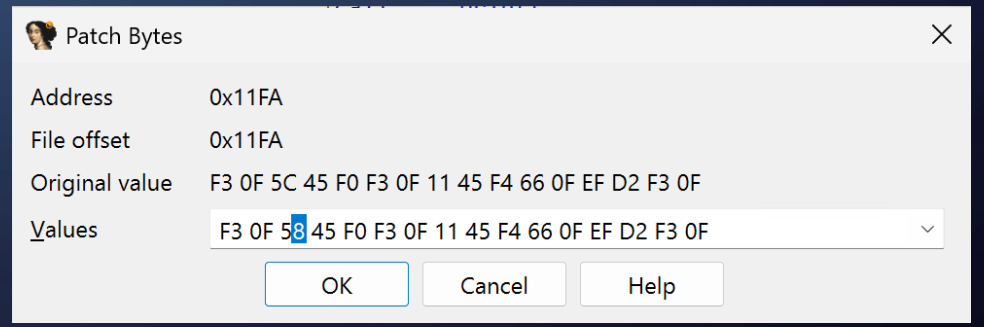
```
call    ___isoc99_scanf
movss   xmm0, [rbp+var_14]
pxor    xmm1, xmm1
cvtss2sd xmm1, xmm0
movsd   xmm0, cs:qword_2060
mulsd   xmm0, xmm1
cvtsd2ss xmm0, xmm0
movss   [rbp+var_10], xmm0
movss   xmm0, [rbp+var_14]
subss   xmm0, [rbp+var_10]
movss   [rbp+var_C], xmm0
```

FIXING THE BUG

- Used:
  - Patch Program > Assemble
- Changed instruction:
  - SUBSS > ADDSS
- Applied patch using Patch Bytes







CODE VERIFICATION

- Checked updated pseudocode:
  - Correct formula used
- Tested program:
  - Input: 100
  - Output:
    - Tax: $7.50
    - Total: $107.50

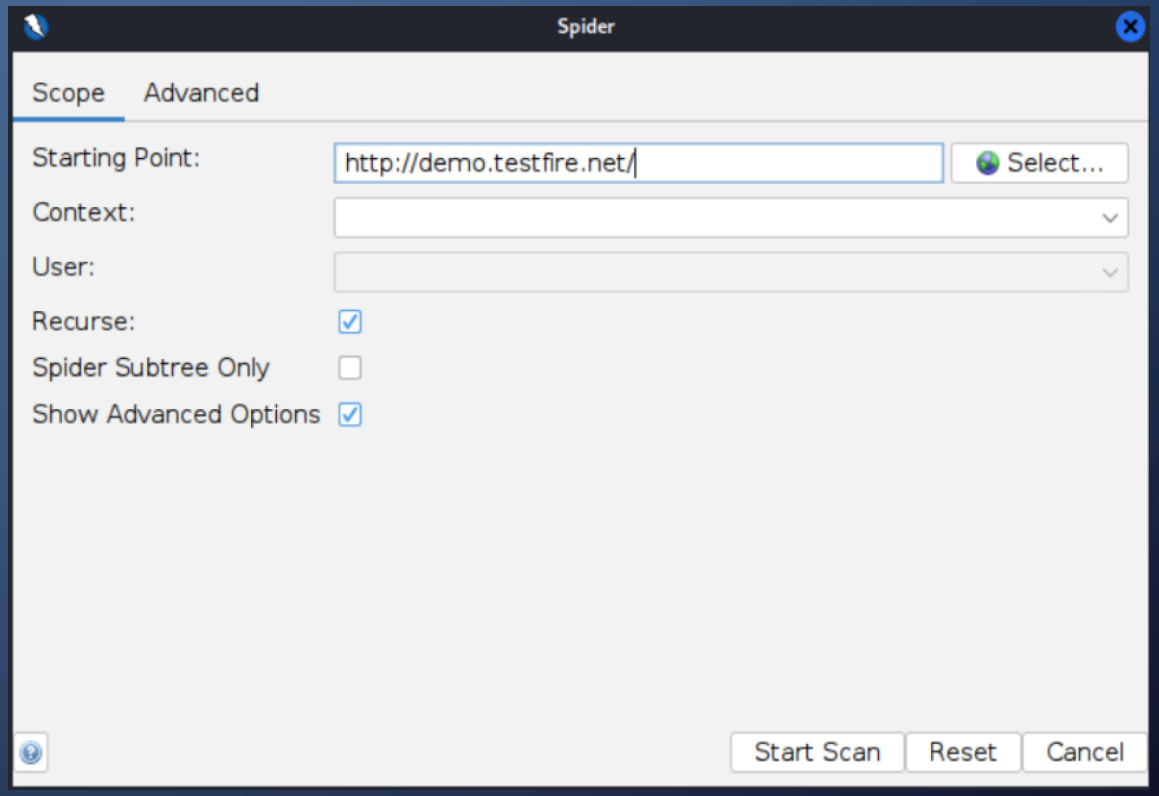
IDA View-A | Pseudocode-A | Strings | Xref Graph: Functior

```
1  int __fastcall main(int argc, const char **argv, const char **envp)
2  {
3    float v3; // xmm0_4
4    float v5[2]; // [rsp+Ch] [rbp-14h] BYREF
5    float v6; // [rsp+14h] [rbp-Ch]
6    unsigned __int64 v7; // [rsp+18h] [rbp-8h]
7
8    v7 = __readfsqword(0x28u);
9    printf("\nEnter in the amount purchased: ");
10   __isoc99_scanf("%f", v5);
11   v3 = 0.075 * v5[0];
12   v5[1] = v3;
13   v6 = v5[0] + v3;
14   printf("\nThe sales tax is $%4.2f", v3);
15   printf("\nThe total bill is $%5.2f\n", v6);
16   return 0;
17 }
```

```
┌──(kali㉿kali)-[~/Downloads]
└─$ ./lab_sales

Enter in the amount purchased: 100

The sales tax is $7.50
The total bill is $107.50

┌──(kali㉿kali)-[~/Downloads]
└─$
```

PART 3 - OWASP ZAP

- Installed ZAP in Kali Linux
- Performed:
  - Spider scan
- Target: http://demo.testfire.net



SCAN REPORT EVALUATION - VULNERABILITIES

Absence of Anti-CSRF Tokens

- Risk - Unauthorized actions
- Attack - User tricked via malicious link
- Mitigation:
  - Add CSRF tokens
  - Validate on server

Content Security Policy (CSP) Header Not Set

- Risk - XSS attacks
- Mitigation:
  - Add Content-Security-Policy header
  - Restrict scripts and sources

SCAN REPORT EVALUATION - VULNERABILITIES

Missing Anti-clickjacking Header

- Risk - Hidden iframe attacks
- Mitigation:
  - Add X-Frame-Options
  - Use frame-ancestors in CSP

Alert type	Risk	Count
Absence of Anti-CSRF Tokens	Medium	3 (27.3%)
Content Security Policy (CSP) Header Not Set	Medium	5 (45.5%)
Missing Anti-clickjacking Header	Medium	5 (45.5%)
Sub Resource Integrity Attribute Missing	Medium	1 (9.1%)

KEY LEARNINGS

- Reverse engineering helps understand binaries
- Small logic errors can cause major issues
- Security testing is essential
- Defense requires multiple layers

CONCLUSION

- Helps to cover the reverse engineering and vulnerability assessment techniques to analyze and improve software security.
- Ghidra, IDA Pro can be used to examine compiled binaries and reconstruct their logic without access to the original source code.
- Highlights the importance of understanding low-level program behavior in debugging and software maintenance.
- OWASP ZAP provides valuable insights into common web application vulnerabilities.
- Important to identify vulnerabilities early and applying effective fixes to ensure system reliability and security.

REFERENCES

- Álvarez Pérez, D., & Tiwari, R. (2025). *Ghidra software reverse-engineering for beginners* (2nd ed.). Packt Publishing.
- Fox, N. (2023, April 6). *How to use Ghidra to reverse engineer malware*. Varonis. https://www.varonis.com/blog/how-to-use-ghidra
- Siahaan, C. N. (2022, October 7). *Binary patching with IDA Pro (part 1)*. Medium. https://medium.com/@k_kisanak/binary-patching-with-ida-pro-part-1-c806d0f20d22
- Minnesota State University Moorhead. (2026). *Installing Ghidra in Kali Linux*. D2L Brightspace. https://mnstate.learn.minnstate.edu/
- Minnesota State University Moorhead. (2026). *Creating and running executable programs in Kali Linux*. D2L Brightspace. https://mnstate.learn.minnstate.edu/
- Medium. (n.d.). *Securing binaries against reverse engineering: A developer's guide*. Medium. https://medium.com/
- Zaproxy. (n.d.). *Getting started*. OWASP ZAP. https://www.zaproxy.org/getting-started/

THANK YOU!